

Just naive Bayes

June Lemieux

2024-12-14

```
# libraries/etc  
library(tidymodels)
```

```
## -- Attaching packages ----- tidymodels 1.2.0 --
```

```
## v broom      1.0.7    v recipes      1.1.0  
## v dials      1.3.0    v rsample      1.2.1  
## v dplyr      1.1.4    v tibble      3.2.1  
## v ggplot2    3.5.1    v tidyr        1.3.1  
## v infer      1.0.7    v tune         1.2.1  
## v modeldata  1.4.0    v workflows    1.1.4  
## v parsnip    1.2.1    v workflowsets 1.1.0  
## v purrr      1.0.2    v yardstick    1.3.1
```

```
## -- Conflicts ----- tidymodels_conflicts() --  
## x purrr::discard() masks scales::discard()  
## x dplyr::filter()  masks stats::filter()  
## x dplyr::lag()     masks stats::lag()  
## x recipes::step()  masks stats::step()  
## * Learn how to get started at https://www.tidymodels.org/start/
```

```
library(discrim)
```

```
## Warning: package 'discrim' was built under R version 4.4.2
```

```
##  
## Attaching package: 'discrim'
```

```
## The following object is masked from 'package:dials':  
##  
##      smoothness
```

```
library(e1071)
```

```
##  
## Attaching package: 'e1071'
```

```
## The following object is masked from 'package:tune':  
##  
##      tune
```

```
## The following object is masked from 'package:rsample':  
##  
##   permutations
```

```
## The following object is masked from 'package:parsnip':  
##  
##   tune
```

```
library(ggplot2)  
library(vip)
```

```
## Warning: package 'vip' was built under R version 4.4.2
```

```
##  
## Attaching package: 'vip'
```

```
## The following object is masked from 'package:utils':  
##  
##   vi
```

```
library(parsnip)  
library(naivebayes)
```

```
## Warning: package 'naivebayes' was built under R version 4.4.2
```

```
## naivebayes 1.0.0 loaded
```

```
## For more information please visit:
```

```
## https://majkamichal.github.io/naivebayes/
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.4.2
```

```
## Loading required package: lattice
```

```
##  
## Attaching package: 'caret'
```

```
## The following objects are masked from 'package:yardstick':  
##  
##   precision, recall, sensitivity, specificity
```

```
## The following object is masked from 'package:purrr':  
##  
##   lift
```

```
library(ROCR)
```

```
## Warning: package 'ROCR' was built under R version 4.4.2
```

```
options(scipen=4)
```

```
## Data Processing
```

```
# Initial data import
```

```
customer_data <- read.csv("~/00 Merrimack/DSE6111/Course Project/customer_data_1.csv", header = TRUE)
```

```
customer_data$Attrition_Flag <- ifelse(customer_data$Attrition_Flag == "Attrited Customer", 1, 0)  
customer_data$Attrition_Flag <- factor(customer_data$Attrition_Flag)
```

```
# check levels
```

```
levels(customer_data$Attrition_Flag)
```

```
## [1] "0" "1"
```

```
# set seed
```

```
set.seed(10)
```

```
# get rid of ClientNum identifier
```

```
customer_data <- customer_data[ -c(1) ]
```

```
# create data split
```

```
data_split <- initial_split(customer_data, strata = "Attrition_Flag", prop = 0.75)
```

```
# create training and test sets
```

```
training_set <- training(data_split)
```

```
test_set <- testing(data_split)
```

```
# row count per set
```

```
nrow(training_set)
```

```
## [1] 7595
```

```
nrow(test_set)
```

```
## [1] 2532
```

```
# use cv folds and tune
```

```
cv_set <- vfold_cv(training_set, v = 10)
```

```
# create recipe for preprocessing
```

```
attrition_recipe_nbayes <-
```

```
  recipe(  
    Attrition_Flag ~ .,  
    data = training_set,  
  ) %>%
```

```
  step_dummy(all_nominal_predictors()) %>%
```

```
  step_dummy(all_nominal_predictors()) %>%
```

```

step_center(all_predictors()) %>%
step_scale(all_predictors())

# create workflow w/o model
nbayes_workflow <- workflow() %>%
  add_recipe(attrition_recipe_nbayes)

# model with nbayes
nbayes_spec <-
  naive_Bayes() %>%
  set_mode("classification") %>%
  set_engine("naivebayes")

# fit this workflow with our training data
nbayes_fit <- nbayes_workflow %>%
  add_model(nbayes_spec) %>%
  fit(data = training_set)

```

```

nb2_wf <- workflow() %>%
  add_recipe(attrition_recipe_nbayes) %>%
  add_model(nbayes_spec)

nb2_rs <- fit_resamples(
  nb2_wf,
  cv_set,
  control = control_resamples(save_pred = TRUE)
)

```

```

#save metrics & preds
nb_rs_metrics <- collect_metrics(nb2_rs)
nb_rs_predictions <- collect_predictions(nb2_rs)

nb_rs_metrics

```

```

## # A tibble: 3 x 6
##   .metric      .estimator mean      n std_err .config
##   <chr>       <chr>    <dbl> <int>   <dbl> <chr>
## 1 accuracy    binary     0.845    10 0.00354 Preprocessor1_Model11
## 2 brier_class binary     0.145    10 0.00348 Preprocessor1_Model11
## 3 roc_auc     binary     0.964    10 0.00169 Preprocessor1_Model11

```

```

# confusion matrix
conf_mat_resampled(nb2_rs, tidy = FALSE)

```

```

##      0      1
## 0 637.3 117.9
## 1   0.2   4.1

```

```

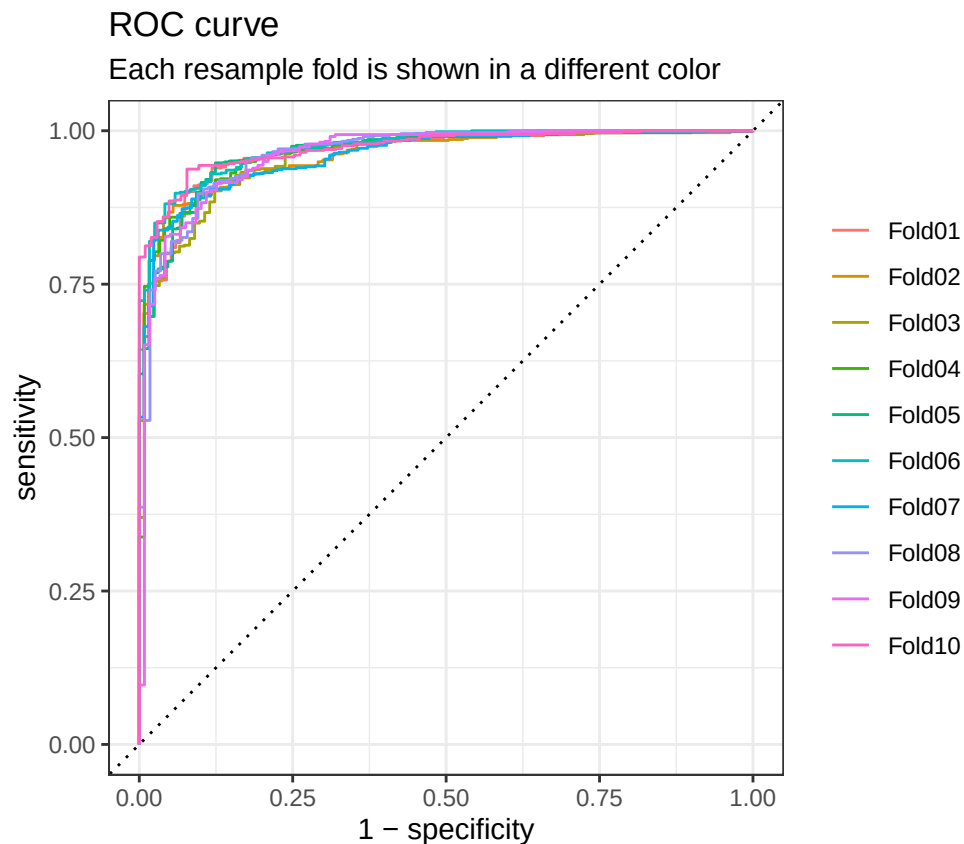
# roc_auc and accuracy
nb2_rs %>% show_best(metric = "roc_auc")

```

```
## # A tibble: 1 x 6
##   .metric .estimator mean      n std_err .config
##   <chr>   <chr>      <dbl> <int>  <dbl> <chr>
## 1 roc_auc binary      0.964    10 0.00169 Preprocessor1_Model1
```

```
best_auc <- nb2_rs %>% select_best(metric = "roc_auc")
best_acc <- nb2_rs %>% select_best(metric = "accuracy")
```

```
# plot
nb_rs_predictions %>%
  group_by(id) %>%
  roc_curve(truth = Attrition_Flag, .pred_0) %>%
  autoplot() +
  labs(
    color = NULL,
    title = "ROC curve ",
    subtitle = "Each resample fold is shown in a different color"
  )
```



```
final_nb_workflow <- finalize_workflow(
  nb2_wf,
  best_auc
)
```

```
# the last workflow
last_nb_fit <-
```

```

final_nb_workflow %>%
  last_fit(data_split)

nb_metrics <- last_nb_fit %>% collect_metrics()

# compare to the null model
null_classification <- null_model() %>%
  set_engine("parsnip")%>%
  set_mode("classification")

null_rs <- workflow() %>%
  add_recipe(attrition_recipe_nbayes) %>%
  add_model(null_classification) %>%
  fit_resamples(
    cv_set
  )

null_metrics <- null_rs %>% collect_metrics()

fimp_df = data.frame(usekernel = TRUE, laplace = 0, adjust = 1)

impModel = train(Attrition_Flag ~ Months_on_book + Total_Relationship_Count +
  Months_Inactive_12_mon + Contacts_Count_12_mon + Total_Amt_Chng_Q4_Q1 +
  Total_Trans_Amt + Total_Trans_Ct + Total_Ct_Chng_Q4_Q1 +
  Avg_Utilization_Ratio,
  data = test_set,
  method = "naive_bayes",
  trControl = trainControl(method="none"),
  tuneGrid = fimp_df)

varImp(impModel)

## ROC curve variable importance
##
##               Importance
## Total_Trans_Ct      100.00
## Total_Ct_Chng_Q4_Q1   84.35
## Avg_Utilization_Ratio  66.20
## Total_Trans_Amt       62.35
## Contacts_Count_12_mon  53.52
## Months_Inactive_12_mon 41.45
## Total_Relationship_Count 31.88
## Total_Amt_Chng_Q4_Q1   27.19
## Months_on_book         0.00

##### plot ROC curve #####

test_results <-
  test_set %>%
  select(Attrition_Flag) %>%
  bind_cols(
    last_nb_fit %>% collect_predictions() %>%

```

```

    select(p_1 = .pred_1)
  )

roc_data <- data.frame(threshold=seq(1,0,-0.01), fpr=0, tpr=0)
for (i in roc_data$threshold) {

  over_threshold <- test_results[test_results$p_1 >= i, ]

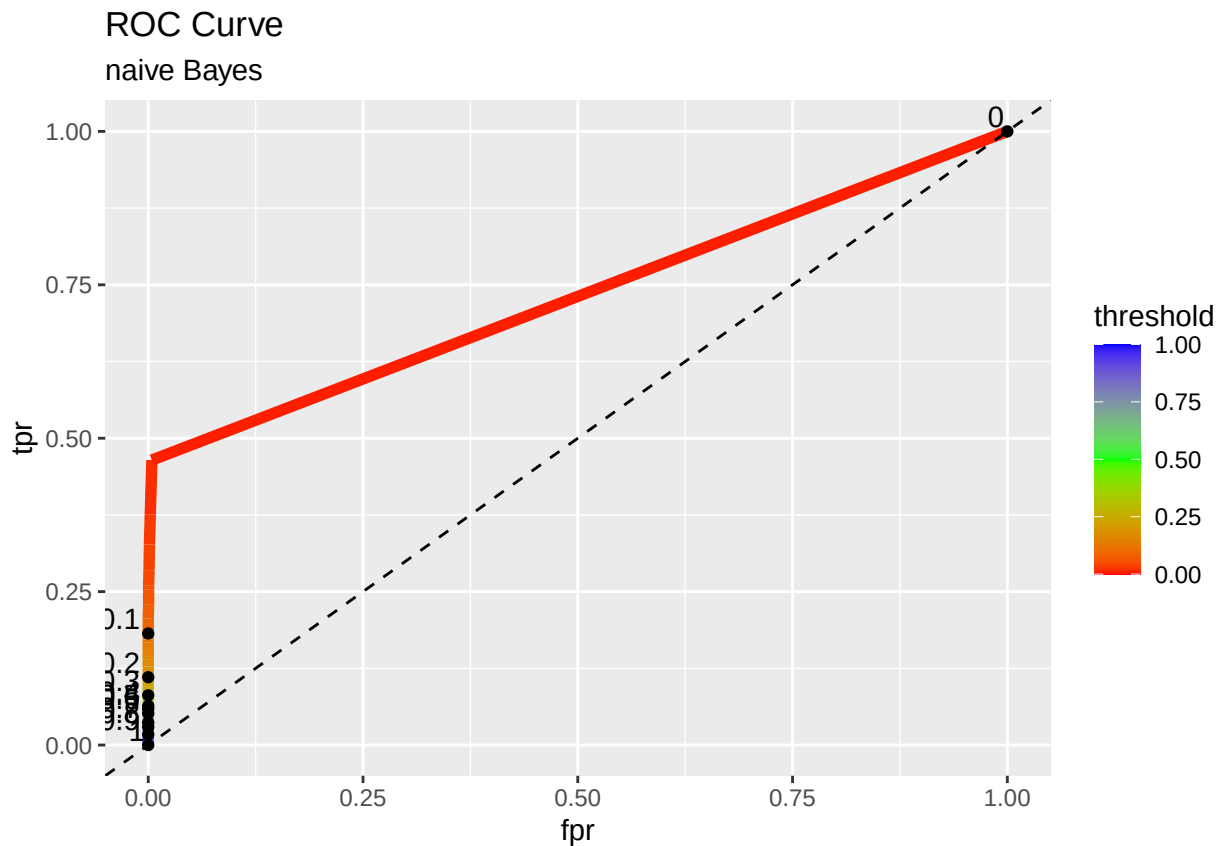
  fpr <- sum(over_threshold$Attrition_Flag==0)/sum(test_results$Attrition_Flag==0)
  roc_data[roc_data$threshold==i, "fpr"] <- fpr

  tpr <- sum(over_threshold$Attrition_Flag==1)/sum(test_results$Attrition_Flag==1)
  roc_data[roc_data$threshold==i, "tpr"] <- tpr

}

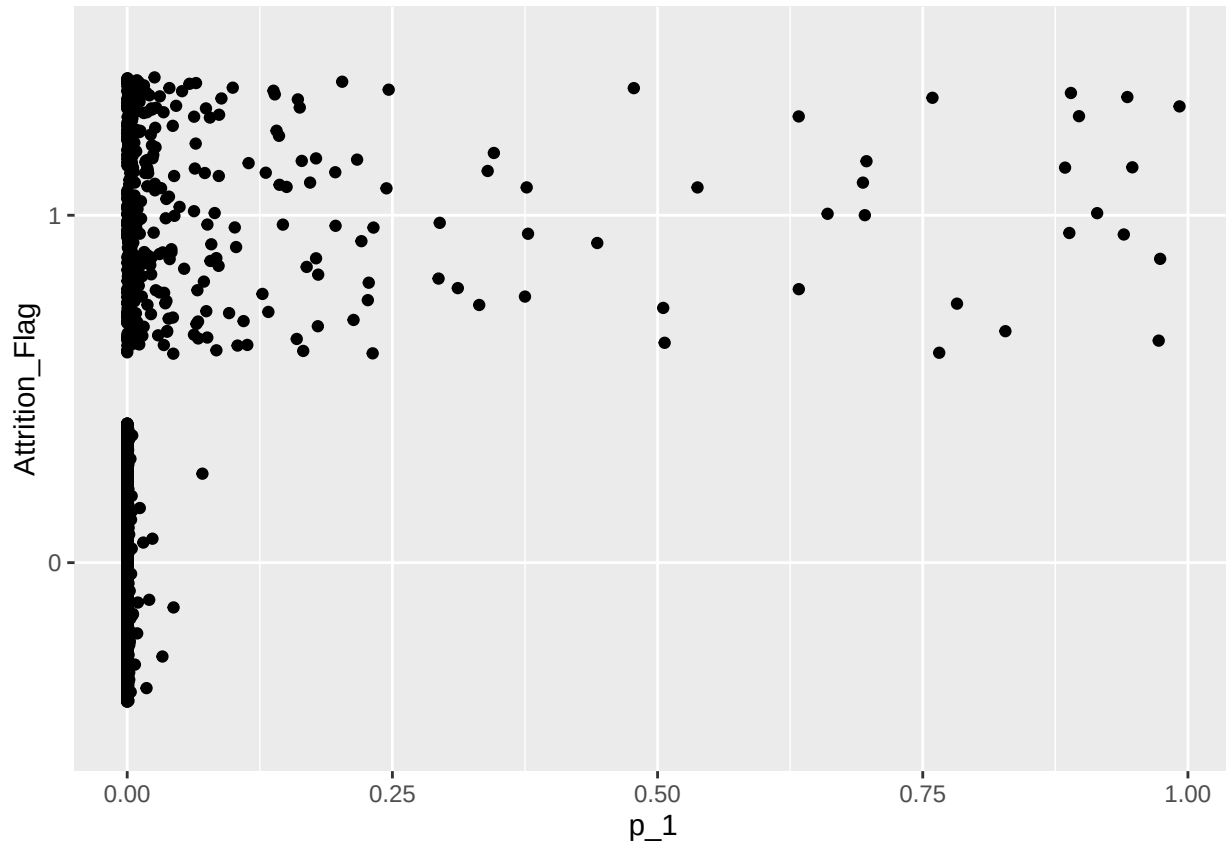
ggplot() +
  geom_line(data = roc_data, aes(x = fpr, y = tpr, color = threshold), linewidth = 2) +
  scale_color_gradientn(colors = rainbow(3)) +
  geom_abline(intercept = 0, slope = 1, lty = 2) +
  geom_point(data = roc_data[seq(1, 101, 10), ], aes(x = fpr, y = tpr)) +
  geom_text(data = roc_data[seq(1, 101, 10), ],
            aes(x = fpr, y = tpr, label = threshold, hjust = 1.2, vjust = -0.2)) +
  ggtitle("ROC Curve", subtitle = "naive Bayes")

```



```
##### ROC curve calculation breakdown #####
```

```
ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +  
  geom_jitter()
```



```
threshold <- 0.25
```

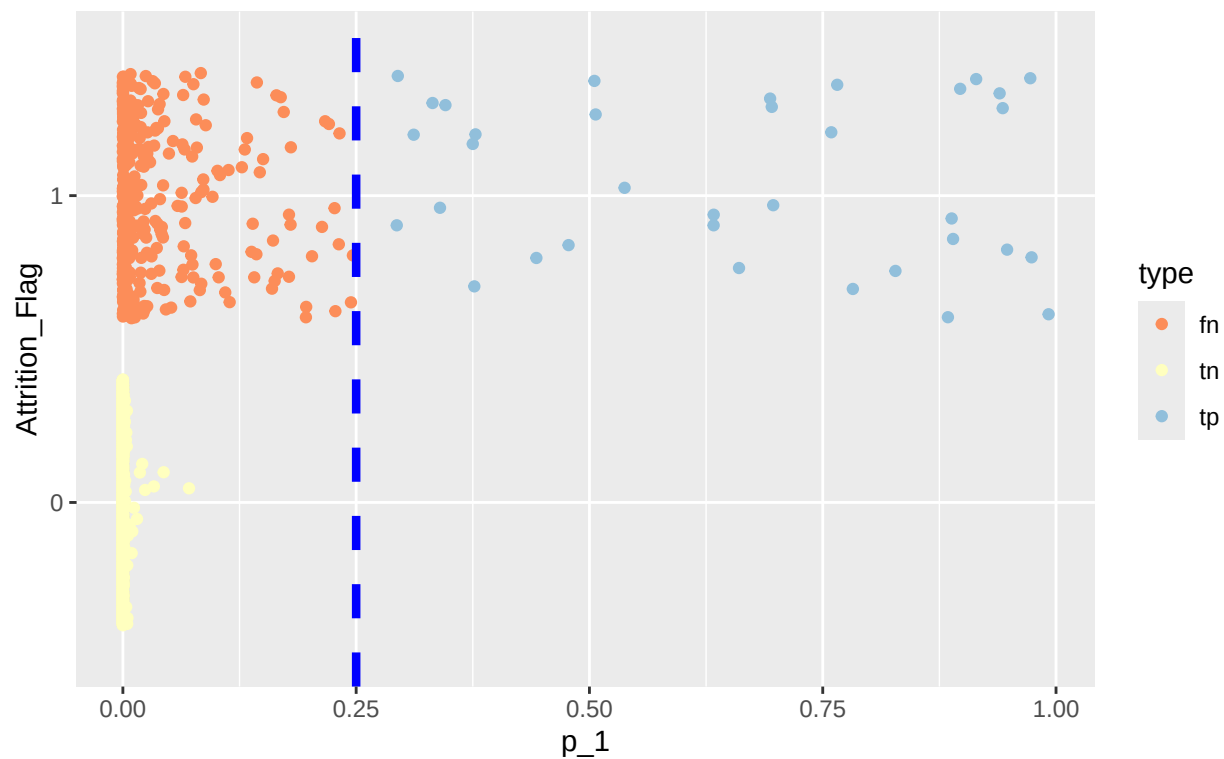
```
test_results$predictions <- ifelse(test_results$p_1 >= threshold, 1, 0)  
tp <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, ])  
fp <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, ])  
tn <- nrow(test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, ])  
fn <- nrow(test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, ])
```

```
test_results$type <- ""  
test_results[test_results$Attrition_Flag==1 & test_results$predictions==1, "type"] <- "tp"  
test_results[test_results$Attrition_Flag==0 & test_results$predictions==1, "type"] <- "fp"  
test_results[test_results$Attrition_Flag==0 & test_results$predictions==0, "type"] <- "tn"  
test_results[test_results$Attrition_Flag==1 & test_results$predictions==0, "type"] <- "fn"
```

```
ggplot(data = test_results, aes(x = p_1, y = Attrition_Flag)) +  
  geom_jitter(aes(colour = type)) +  
  geom_vline(xintercept = threshold, linetype = "dashed", color = "blue", linewidth = 1.5) +  
  scale_color_brewer(palette = "RdYlBu") +  
  ggtitle("ROC Curve Calculation Breakdown", subtitle = "naive Bayes")
```


ROC Curve Calculation Breakdown

naive Bayes



```
fpr <- fp/(fp + tn)
tpr <- tp/(tp + fn)
```

```
# confusion matrix data
tn
```

```
## [1] 2125
```

```
fn
```

```
## [1] 372
```

```
fp
```

```
## [1] 0
```

```
tp
```

```
## [1] 35
```

```
““
```